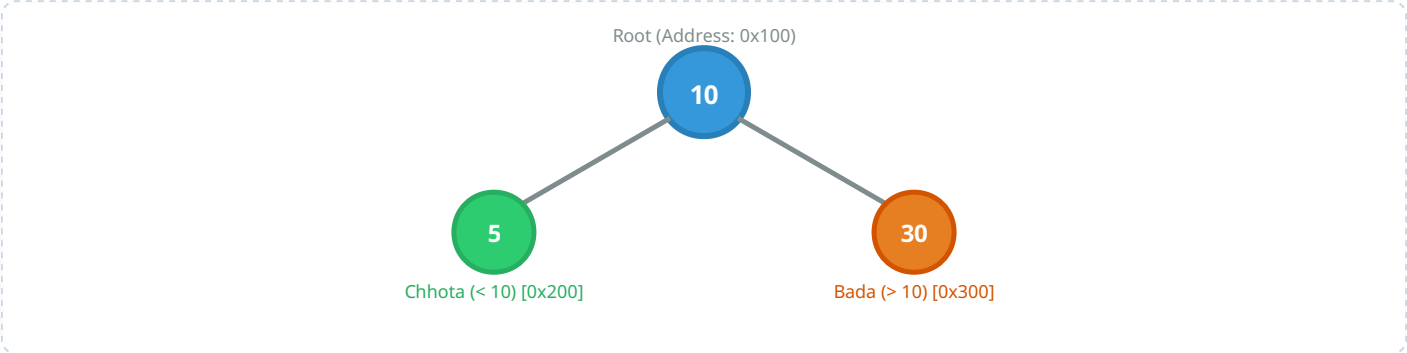
Binary Search Tree (BST): Deletion & Memory Link 🌳

Chai aur Code DSA Series • Visual Illustrated Notes (Beginner & Kid Friendly)

1. BST Kya Hota Hai? (Easy Analogy & Memory Concept) 🧠

BST ko ek aise **family tree / school line** ki tarah samjho jahan hamesha ek strict rule follow hota hai:

- **Left Child:** Apne parent node se hamesha **Chhota (Smaller)** hoga.
- **Right Child:** Apne parent node se hamesha **Bada (Greater)** hoga.



💾 **Memory Link (Heap & Pointers):**
Har ek Node computer memory (RAM Heap) mein ek **Object box** hota hai jisme 3 cheezein hoti hain:
`[ left_pointer | data | right_pointer ]`
Jab hum `root.left = new_node` karte hain, toh Parent box ke andar Left Child ka **Memory Address** save ho jata hai!

2. Inorder Traversal, Successor aur Predecessor 🎯

Magic Property of BST: Jab aap BST ka **Inorder Traversal (Left → Root → Right)** nikaalte ho, toh saare elements hamesha **Sorted (Increasing Order)** mein aate hain!

👑 Inorder Successor (Next Bada Element)

Sorted order mein kisi node ke **theek baad** aane wala number.

BST mein dhoondhne ka Secret Rule:

1. 1 step **RIGHT** jao 👉
2. Phir jab tak possible ho, **LEFT** jate jao 👈 (Left-most element).

Example: 30 ka successor = 30 ke right (40) ke left-most (32).

🛡️ Inorder Predecessor (Pichhla Chhota Element)

Sorted order mein kisi node se **theek pehle** aane wala number.

BST mein dhoondhne ka Secret Rule:

1. 1 step **LEFT** jao 👈
2. Phir jab tak possible ho, **RIGHT** jate jao 👉 (Right-most element).

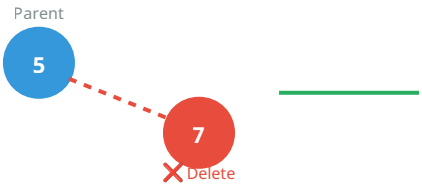
Example: 30 ka predecessor = 30 ke left (25) ka right-most element.

3. BST Deletion ke 3 Master Cases ✂️

Jab bhi hume kisi node ko delete karna hota hai, computer memory mein pointer manipulation ke 3 cases bante hain:

Case 1 0 Child (Leaf Node Deletion) 🍃

Situation: Node akela hai, uska koi bachha nahi hai (Leaf Node). Jaise Node 12 ya 7.



> 🧠 **Memory Action:** Parent ko sidha None (Null pointer) return kar do! Parent ka link toot gaya, garbage collector memory free kar dega.

Case 2 1 Child (Single Child Node Deletion) 👶

Situation: Node ka sirf ek bachha hai (ya toh left ya right). Jaise Node 40 jiska child 50 hai.

> 🧠 **Memory Action (Grandparent Responsibility):** Node 40 delete hote hi apna bacha (50) apne parent (30) ko handover kar deta hai: `30.right = 50`!

Case 3 2 Children (Both Left & Right Exist) 👨‍👩‍👧

Situation: Node ke dono bachhe hain (e.g. Node 30). Hum directly node ko nahi mita sakte varna pura ped bikhar jayega!

> **2-Step Master Strategy:**
>
> **Step 1 (Copy Value):** 30 ka *Inorder Successor* (32) dhoondho aur 32 ki value ko 30 ke dabbe me **Copy / Overwrite** kar do.
>
> **Step 2 (Delete Successor):** Ab original 32 (jo niche leaf/single-child hai) usko delete karne ke liye right subtree me recursion call kardo!

4. Complete Python Implementation & Memory Call Stack 💻

```python
# Node Structure in Memory (Heap Allocation)
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None   # Memory pointer to left child
        self.right = None  # Memory pointer to right child

# Helper function to find Inorder Successor (1-Right, then Left-most)
def get_successor(curr):
    curr = curr.right
    while curr is not None and curr.left is not None:
        curr = curr.left
    return curr

# Main BST Deletion Function
def delete_node(root, val):
    # Base Case 1: Empty Tree
    if root is None:
        return None

    # Search phase (Recursion traversal)
    if val < root.data:
        root.left = delete_node(root.left, val)
    elif val > root.data:
        root.right = delete_node(root.right, val)
    else:
        # Target found! Handle Cases 1 & 2
        if root.left is None:
            return root.right  # Returns None (Case 1) or Right Child (Case 2)
        elif root.right is None:
            return root.left   # Returns Left Child (Case 2)

        # Handle Case 3 (2 Children)
        succ = get_successor(root)
        root.data = succ.data  # Overwrite value
        root.right = delete_node(root.right, succ.data) # Delete duplicate

    return root
```

5. Quick Revision & Memory Flow Summary ⚡

Operation / Scenario	Memory Pointer Action	Time Complexity	Space Complexity (Stack)
Search Target Node	Traverse left if smaller, right if greater	O(h) → O(log N) balanced	O(h) recursive call stack
Delete 0 Child (Leaf)	Parent pointer gets updated to None	O(1) after search	O(1) extra space
Delete 1 Child	Parent directly connects to child (bypass node)	O(1) after search	O(1) extra space
Delete 2 Children	Copy successor data + recursively delete leaf/ single	O(h) for successor search	O(h) stack frames

🎉 **Golden Rule to Remember:** BST me deletion ka secret hai — *"Pointer link ko Parent ke sath attach karo, aur 2 children ke case me successor ko copy karke duplicate delete karo!"*